

Mesterséges Intelligenciák alapjai

II. beadandó – Genetikus algoritmusok

Név: Gál Olivér István Neptun: ISXBF3

2025. november 25.

1. Bevezetés

Ebben a beadandóban genetikus algoritmust (GA) használok három klasszikus folytonos tesztfüggvény (Rastrigin, Booth, Lévi) minimalizálására, majd a paraméterek (generációszám, populációméret, lépésméret, szelekciós és crossover módszerek, elitizmus, a mutáció intenzitása és a keresztezés módja) hatását vizsgálom. A kód Pythonban, NumPy segítségével készült, a vizualizációhoz matplotlib-et használok.

A dokumentum a Rastrigin-, Booth- és Lévi-függvényeken végzett kísérleteket, valamint egy kiterjedt paraméter-söprést tartalmaz. Végül a genetikus algoritmust az utazó ügynök problémára (TSP) is alkalmazom 10 és 20 városos példákon..

2. Folytonos tesztfüggvények

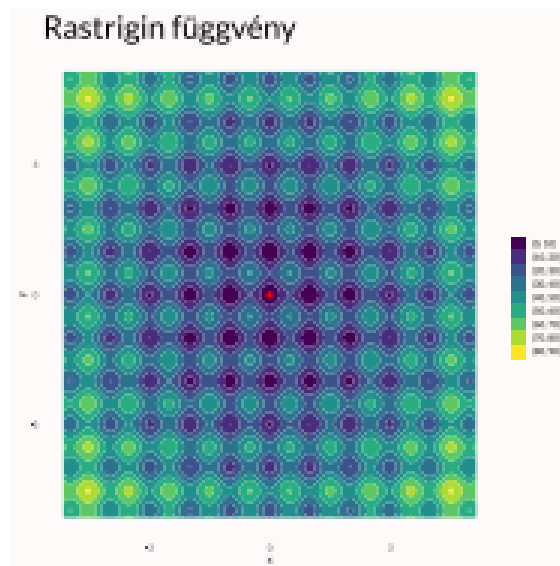
2.1. Rastrigin függvény

A Rastrigin-függvény n dimenzióban:

$$f(\mathbf{X}) = An + \sum_{i=1}^n [x_i^2 - A \cos(2\pi x_i)] , \quad (1)$$

ahol $A = 10$, a tartomány $-5,12 \leq x_i \leq 5,12$, és a globális minimum

$$f(\mathbf{0}) = f(0, \dots, 0) = 0.$$



1. ábra. Rastrigin függvény

1. ábra. Rastrigin függvény (kontúrábra, $n = 2$).

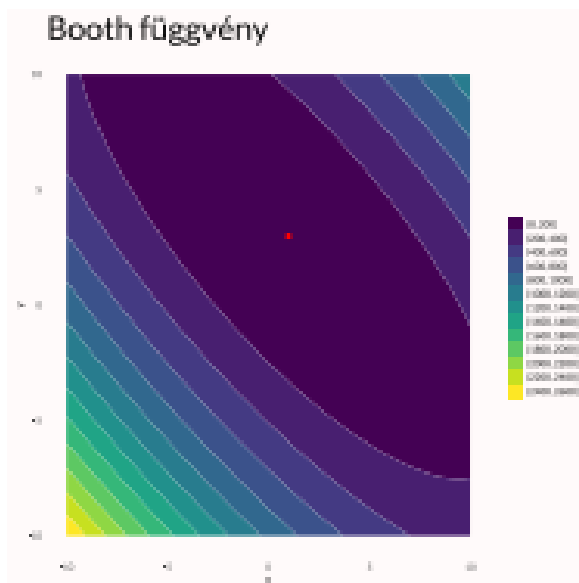
2.2. Booth függvény

A Booth-függvény két dimenzióban:

$$f(x, y) = (x + 2y - 7)^2 + (2x + y - 5)^2, \quad (2)$$

tartomány: $-10 \leq x, y \leq 10$, globális minimum:

$$f(1, 3) = 0.$$



2. ábra. Booth függvény

2. ábra. Booth függvény (kontúrábra).

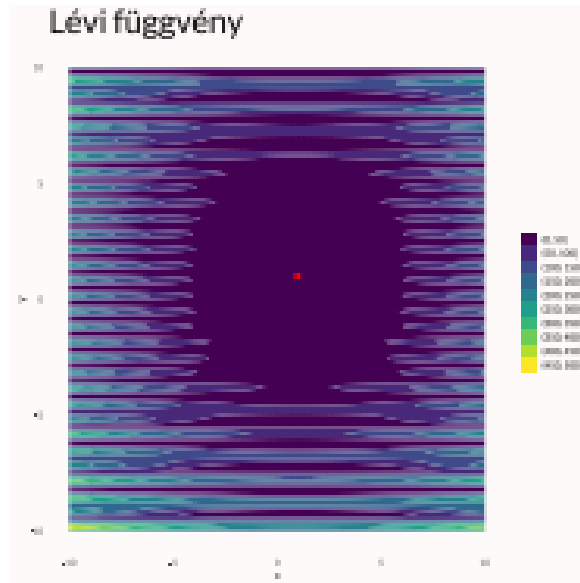
2.3. Lévi függvény

A Lévi-függvény két dimenzióban:

$$f(x, y) = \sin^2(3\pi x) + (x - 1)^2 (1 + \sin^2(3\pi y)) + (y - 1)^2 (1 + \sin^2(2\pi y)), \quad (3)$$

tartomány: $-10 \leq x, y \leq 10$, globális minimum:

$$f(1, 1) = 0.$$



3. ábra. Lévi függvény

3. ábra. Lévi függvény (kontúrábra).

3. Genetikus algoritmus felépítése

A GA implementáció a `continuous.ga_core.GeneticAlgorithm` osztályban található. A főbb elemek:

- **Reprezentáció:** egy egyed egy valós vektor $\mathbf{x} \in \mathbb{R}^d$ (`numpy.ndarray`), ahol d a dimenzió.
- **Inicializálás:** az egyedek minden komponense egyenletesen kerül kisorsolásra a megadott tartományon belül.
- **Célfüggvény:** a `cost_function` paraméterrel adott függvény (pl. `rastrigin`, `booth`, `levi`), amelyet *minimalizálni* szeretnénk.
- **Szelekciós módszerek:**
 - *Versenyeztetés* (tournament),

- rulettekerék (roulette),
 - rangsor-szerinti (rank),
 - fitnessz-rangsor szerinti (fitness_rank),
 - diverzitás alapú (diversity).
- **Keresztezés (crossover):**
 - egyponos (one_point),
 - kétpontos (two_point),
 - többpontos (k_point),
 - uniform,
 - *path relinking* (a két szülő közti szakaszon választott pontokkal).
 - **Mutáció:** minden génre függetlenül, Gauss-zajjal (szórás: `step_size`), majd a vektor vágása a megadott tartományra.
 - **Elitizmus:** ha be van kapcsolva, a legjobb egyed változatlanul átkerül a következő generációba.
 - **Leállási feltétel:** alapértelmezetten fix generációs szám (`generations`), de opcionálisan *korai leállást* is használok: ha a legjobb érték egy adott tolerancia alá csökken, vagy ha bizonyos számú generáción keresztül nem javul a legjobb egyed (`tolerance`, `patience` paraméterek).

4. Kódstruktúra röviden

A beadandó forráskódja egy egyszerű mappastruktúrában helyezkedik el:

- `beadando.tex`: a jelen dokumentum L^AT_EX forrása.
- `beadando.pdf`: a lefordított beadandó.
- `main_continuous.py`: belépési pont a folytonos tesztfüggvényeken (főleg 2D Rastrigin) futtatott alap kísérletekhez. Itt hasonlítom össze a különböző szelekciós módokat és crossover operátorokat, és innen hívom meg a `GeneticAlgorithm` osztályt.

A kontinuos GA-hoz tartozó modulok a `continuous/` mappában vannak:

- `continuous/ga_core.py`: a genetikus algoritmus magja. Itt található a `GeneticAlgorithm` osztály, amely
 - inicializálja a populációt a megadott tartományon,
 - kezeli az összes szelekciós módszert (tournament, roulette, rank, fitness_rank, diversity),
 - megvalósítja az összes crossover típust (egyponos, kétpontos, `k_point`, uniform, `path_relink`),
 - elvégzi a Gauss-zajos mutációt és a határok közé vágást,

- elitizmust és opcionális *early stopping* feltételeket kezel (tolerance, patience),
 - minden generációban naplózza a legjobb, legrosszabb, átlagos értéket és a szórást.
- `continuous/functions.py`: a tesztfüggvények (Rastrigin, Booth, Lévi) implementációja és a hozzájuk tartozó tartomány-információk. Ezt a modult használja minden folytonos kísérlet.
 - `continuous/experiments.py`: a paraméter-söprést végző szkript. Végigiterál a generációs szám, populációméret, lépésméret, szelekciós mód, crossover és elitizmus kombinációin, futtatja a GA-t, és az eredményeket egy közös CSV-fájlba írja.
 - `continuous/table_gen.py`: a paraméter-söprés eredményeit tartalmazó CSV-t (`continuous_param_sweep.csv`) olvassa be, kiválasztja a legjobb konfigurációkat, és LaTeX táblázatokat generál (`results_rastrigin.tex`, `results_booth.tex`, `results_levi.tex`, valamint az összefoglaló `results_table.tex`).
 - `continuous/plots.py`: a konvergencia-vizsgálatokhoz tartozó ábrákat készíti. Beolvassa a GA által eltárolt történeti adatokat, és előállítja a `conv_rastrigin_good.png`, `conv_rastrigin_bad.png` és `conv_rastrigin_compare.png` konvergencia-grafikonokat (legjobb, legrosszabb, átlag, szórás).
 - `continuous/__init__.py`: üres inicializáló fájl, amely lehetővé teszi, hogy a `continuous` mappát Python-modul formájában importáljam.

A futások eredményeit és az ábrákat tartalmazó állományok:

- `continuous_param_sweep.csv`: a folytonos tesztfüggvényeken végzett teljes paraméter-söprés nyers eredménye (minden sor egy paraméterkészlet: G , K , L , szelekció, crossover, elitizmus, célfüggvény, legjobb érték, futási idő).
- `results_table.tex`: összefoglaló LaTeX táblázat a legjobb konfigurációkról, amelyet közvetlenül beillesztek a beadandóba.
- `images/`: minden, a dokumentumban használt ábra: a tesztfüggvények kontúrarábrái (`rastring.png`, `booth.png`, `levi.png`) és a konvergencia-grafikonok (`conv_rastrigin_*.png`).

A diszkrét TSP-esetekhez külön mappát tartok fenn:

- `tsp/`: az utazó ügynök problémához használt genetikus algoritmus és a hozzá tartozó kísérleti szkriptek. A folytonos esettől függetlenül, permutációs kódolással valósítják meg a GA-t:
 - `tsp/tsp_ga.py`: a TSP-re írt genetikus algoritmus megvalósítása. A `TSPGeneticAlgorithm` osztály kezeli a permutációs reprezentációt, a tournament/roulette szelekciót, az OX/PMX keresztezést, a swap mutációt, valamint az elitizmust és a konvergencia-naplózást.

- `tsp/tsp_instances.py`: a 10 és 20 városos tesztinstance-ok definíciója. 2D koordinátalistákból szimmetrikus távolság-mátrixot épít (`build_distance_dict`), és ezeket `SMALL_10_CITY` illetve `MEDIUM_20_CITY` néven exportálja.
- `tsp/tsp_experiments.py`: a TSP-kísérletek futtatására szolgáló szkript. A `run_single_tsp` függvény állítja be a GA paramétereit, elindítja a `TSPGeneticAlgorithm`-ot, majd kiírja a legjobb körhosszt és a futási időt a 10 és 20 városos instance-okra.

5. Kísérletek Rastrigin függvényen (2D)

A Rastrigin esetén a fő kísérleteket $d = 2$ dimenzióban végeztem. Az alapbeállítás a következő volt:

$$K = 100, \quad G = 200, \quad p_c = 0,9, \quad p_m = 0,3, \quad L = 0,5.$$

ahol K a populációméret, G a generációk száma, p_c a `crossover_rate`, p_m a `mutation_rate`, L pedig a `step_size` (mutáció lépésmérete).

5.1. Szelekciós módszerek és crossover operátorok

A `main_continuous.py` fájlban külön lefutatom a különböző szelekciós módokat (`tournament`, `roulette`, `rank`, `fitness_rank`, `diversity`), majd fix tournament szelekció mellett a különböző crossover operátorokat (`one_point`, `two_point`, `k_point`, `uniform`, `path_relink`). A futások eredményeként mindegyik esetben a Rastrigin globális minimuma körüli pontokat talál az algoritmus, nagyon kicsi célfüggvény-értékkel.

5.2. Paraméter-söprés és eredménytáblázat

A paraméter-söprést az `experiments.py` végzi. A következő paramétereket kombináltam:

- generációs szám $G \in \{5, 10, 20, 50, 100\}$,
- populációméret $K \in \{5, 10, 20, 50, 100\}$,
- lépésméret $L \in \{0,1, 0,2, 0,5, 1,0, 1,5, 2,0\}$,
- ötféle szelekciós módszer,
- ötféle crossover operátor,
- elitizmus: be- és kikapcsolva.

A legjobb (legkisebb célfüggvény-értékű) 10 konfigurációt a `table_gen.py` egy LaTeX táblázatba írja.

G	K	L	Val. fv.	Elit	Célfv.	Eredmény	t
100	50	0.1	tournament, path_relink	Igen	Rastrigin	$f([-5.19627\text{e-}07 \text{ } -7.38487\text{e-}07]) = 1.618\text{e-}10$	0.5149 s
100	100	0.1	tournament, path_relink	Igen	Rastrigin	$f([-7.29957\text{e-}08 \text{ } 9.46612\text{e-}07]) = 1.788\text{e-}10$	0.8562 s
50	100	0.2	fitness_rank, uniform	Igen	Rastrigin	$f([1.94972\text{e-}06 \text{ } 1.32459\text{e-}06]) = 1.102\text{e-}09$	2.4773 s
100	100	0.2	fitness_rank, uniform	Igen	Rastrigin	$f([1.94972\text{e-}06 \text{ } 1.32459\text{e-}06]) = 1.102\text{e-}09$	3.1737 s
100	100	0.2	tournament, path_relink	Igen	Rastrigin	$f([3.11887\text{e-}06 \text{ } 4.34458\text{e-}07]) = 1.967\text{e-}09$	1.0203 s
100	50	2.0	tournament, path_relink	Igen	Rastrigin	$f([2.26096\text{e-}06 \text{ } 2.61728\text{e-}06]) = 2.373\text{e-}09$	0.5568 s
100	50	0.2	tournament, path_relink	Igen	Rastrigin	$f([-5.05097\text{e-}06 \text{ } -8.46454\text{e-}07]) = 5.204\text{e-}09$	0.5633 s
100	20	0.5	rank, path_relink	Igen	Rastrigin	$f([2.44129\text{e-}06 \text{ } -7.05303\text{e-}06]) = 1.105\text{e-}08$	0.3155 s
100	100	0.5	tournament, path_relink	Igen	Rastrigin	$f([-1.21544\text{e-}05 \text{ } 2.51543\text{e-}06]) = 3.056\text{e-}08$	0.9781 s
50	50	0.2	tournament, path_relink	Igen	Rastrigin	$f([-4.34946\text{e-}06 \text{ } 1.30415\text{e-}05]) = 3.750\text{e-}08$	0.2919 s

1. táblázat. A legjobb paraméterkonfigurációk a Rastrigin függvényen (2D).

5.3. Szelekciós módszerek összehasonlítása

A `main_continuous.py` fájlban a Rastrigin-függvényen külön összehasonlítottam az egyes szelekciós módszereket: *tournament*, *roulette* (relatív túlélési valószínűség), *rank*, *fitness_rank* és *diversity*. A kísérletekhez fix paraméterkészletet használtam (2 dimenzió, $K = 100$ kromoszóma, $G = 200$ generáció, $\text{step_size} = 0,5$, $\text{crossover_rate} = 0,9$, $\text{mutation_rate} = 0,3$, elitizmus bekapcsolva), és csak a szelekció módját változtattam, hogy az összehasonlítás tiszta legyen.

Szelekció	Legjobb $f(x)$	Futási idő [s]	Megjegyzés
tournament	$7.387 \cdot 10^{-9}$	0.942	gyors, stabil konvergencia
roulette	$1.158 \cdot 10^{-4}$	1.296	érzékeny a skálázásra
rank	$1.741 \cdot 10^{-6}$	1.721	kiegyensúlyozott szelekció
fitness_rank	$3.372 \cdot 10^{-6}$	2.328	a legjobbkat erősen preferálja
diversity	$1.894 \cdot 10^{-3}$	32.119	nagyobb változatosságot tart fenn

2. táblázat. Szelekciós módszerek összehasonlítása a Rastrigin-függvényen (2D).

A futások azt mutatták, hogy a *tournament* és a *fitness_rank* szelekció általában gyorsan a globális minimum közelébe viszi a populációt, a legjobb $f(x)$ értékek itt lettek a legkisebbek. A sima *rank* szelekció valamivel gyengébb eredményt adott, de cserébe kiegyensúlyozottabban húzza be a teljes populációt a jó régiókba.

A *roulette* (relatív túlélési valószínűséggel) a Rastrigin esetén érzékenyebb volt a cost-értékek skálájára: előfordult, hogy néhány nagyon jó egyed túlságosan dominálta a sorsozást, és a keresés könnyebben ragadt be lokális optimumokba. A *diversity* alapú szelekció ezzel szemben jobban megőrizte a sokféleséget: a legjobb értékei kissé gyengébbek lettek, viszont a populáció eloszlása a keresési térben jóval szórta maradt, ami hasznos lehet nehezebb, sok lokális minimummal rendelkező függvényeknél.

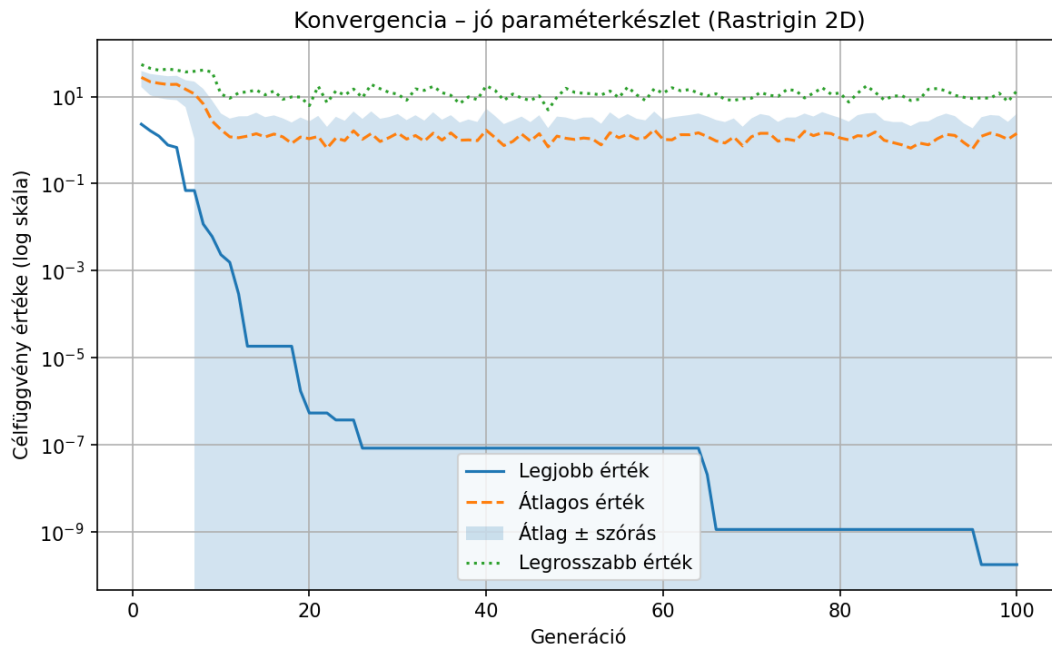
Összességében a Rastrigin-függvényen a *tournament* és a *fitness_rank* adta a legjobb kompromisszumot a megoldás minősége és a futási idő között, ezért a további kísérletekben (keresztésvizsgálat, paraméter-söprés) ezeket használtam alapbeállításként.

5.4. Konvergencia vizsgálata

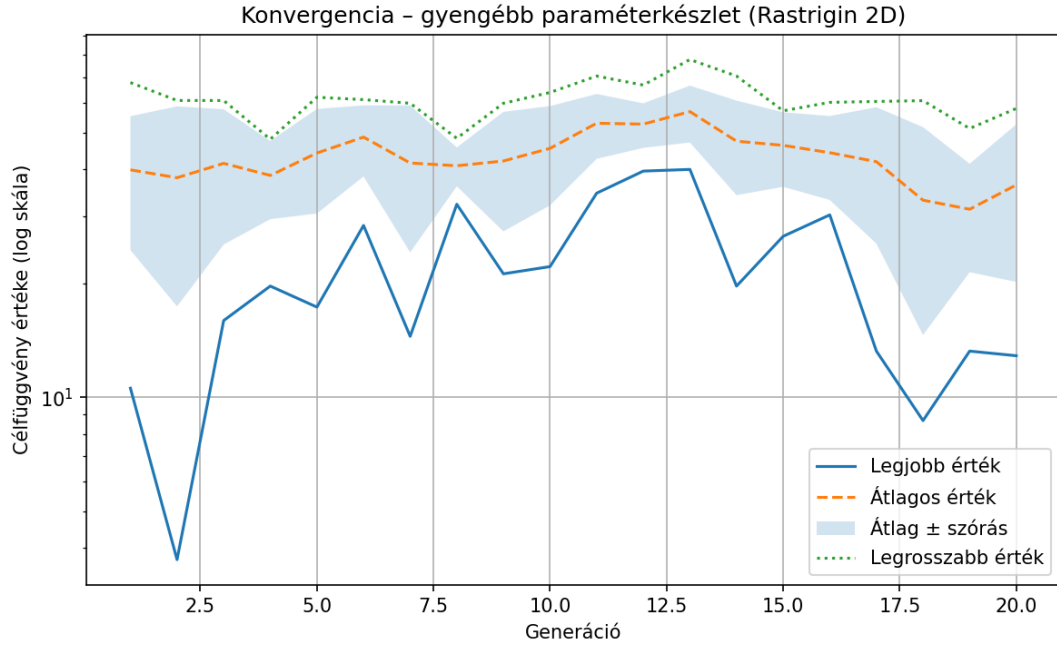
A `plots.py` két különböző paraméterkészletet hasonlít össze: egy „jó” konfigurációt (sok generáció, nagy populáció, kis lépésméret, elitizmus, *tournament* szelekció és

`path_relink` crossover), és egy gyengébb beállítást (keves generáció, kis populáció, nagy lépésméret, elitizmus nélkül, diverzitás alapú szelekcióval).

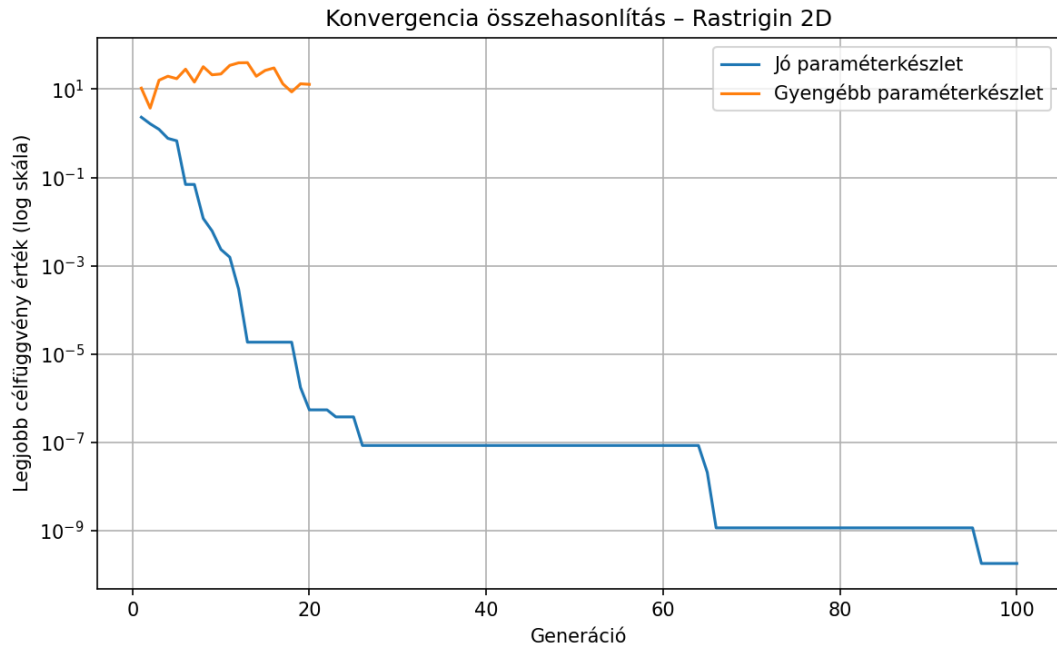
A genetikus algoritmus minden generációban eltárolja *(i)* a legjobb egyed célfüggvény-értékét (*legjobb érték*), *(ii)* a populáció átlagos célfüggvény-értékét, *(iii)* a szórást, valamint *(iv)* a legrosszabb egyed célfüggvény-értékét. A konvergencia-grafikonokon a kék folytonos görbe a legjobb, a narancssárga szaggatott az átlagos értéket mutatja, a kék árnyékolt sáv az átlag $\pm$ szórás tartományt jelöli, a zöld pontozott görbe pedig a *legrosszabb* egyed értékét adja generációnként. Ezzel a feladatkiírásban szereplő „legjobb, legrosszabb, szórás” vizsgálati szempont mind megjelenik.



4. ábra. Konvergencia – jó paraméterkészlet (Rastrigin 2D).



5. ábra. Konvergencia – gyengébb paraméterkészlet (Rastrigin 2D).



6. ábra. Két paraméterkészlet konvergenciájának összehasonlítása (Rastrigin 2D).

A jó paraméterkészletnél a legjobb célfüggvény-érték gyakorlatilag exponenciálisan csökken, és néhány tucat generáció után eléri a globális minimumot ($f(\mathbf{0}) \approx 10^{-9}$ nagyságrendben). Ezzel párhuzamosan az átlag és a legrosszabb érték is fokozatosan közelít a minimumhoz, a szórás pedig csökken: a populáció „összehúzódik” a minimum környékére.

A gyengébb paraméterkészletnél a legjobb érték is nagyságrendekkel rosszabb marad, az átlag és a legrosszabb görbék magasabban futnak, és a szórás is nagyobb. Ez azt

mutatja, hogy kicsi populációval, kevés generációval és elitizmus nélkül az algoritmus nem tud stabilan konvergálni a globális minimum közelébe.

5.5. Az elitizmus hatása

Az elitizmus szerepe jól látható a paraméter-söprés eredményein is: a 1. táblázatban szereplő tíz legjobb konfiguráció mindegyikében be van kapcsolva az elitizmus (*Elit* = *Igen*). Ez azt jelenti, hogy a legjobb egyedek generációnként mindig változatlanul átmentem a következő populációba, így a már megtalált jó megoldás nem „veszhet el” a mutációk és keresztezések miatt.

Elitizmus nélkül a populáció nagyobb szabadsággal tud mozogni az állapottérben, de gyakran előfordul, hogy egy jó megoldást későbbi generációk „lerombolnak”, és a legjobb érték ugrál, vagy akár romlik is. Elitizmussal a legjobb érték monoton nemnövekvő (minimalizálunk), a konvergencia stabilabb, ugyanakkor a populáció sokkal gyorsabban össze tud húzódní egy lokális optimum környezetében. A paraméter-söprés eredményei alapján a Rastrigin-függvényen az elitizmus egyértelműen javította a legjobb elért értéket, miközben a futási időt gyakorlatilag nem befolyásolta.

5.6. A mutáció intenzitásának és a keresztezés módjának hatása

A mutáció erejét az algoritmusban a `step_size` paraméter szabályozza: a Gauss-zaj szórása megegyezik a lépésmérettel, így nagyobb `step_size` esetén a mutációk nagyobb ugrásokat okoznak a keresési térben. A feladatkiírásnak megfelelően a paraméter-söprésben

$$L \in \{0,1,0,2,0,5,1,0,1,5,2,0\}$$

értékeket vizsgáltam. A 1. táblázat alapján a legjobb konfigurációk mind kis lépésmérettel (jellemzően $L = 0,1$ vagy $0,2$) dolgoznak: ezeknél a mutáció inkább finomhangolást végez a globális minimum környezetében, nem „dobálja szét” a már jó egyedeket. Nagy lépésméret ($L \geq 1,5$) mellett a célfüggvény-értékek lényegesen rosszabbak maradtak, ami azt jelzi, hogy a túl erős mutáció zajossá teszi a keresést.

A rekombináció hatását több különböző keresztezési operátorral vizsgáltam: egypon-tos, kétpontos, többpontos (`k_point`), uniform, illetve *path relinking* crossoverrel. A paraméter-söprés top eredményei alapján a `path_relink` és a `uniform` crossover szerepel leggyakrabban a legjobb konfigurációk között, míg az egyszerű egypon-tos keresztezés ritkábban kerül az élmezőnybe. Ez arra utal, hogy a Rastrigin-függvényen előnyös, ha a rekombináció nem csak egy fix vágási pont körül cserél géneket, hanem a két szülő közötti szakaszon, illetve génszinten „összekeverve” tud új egyedeket létrehozni.

Összefoglalva: a mérések alapján a Rastrigin-függvényen egy olyan konfiguráció működik jól, ahol

- a mutáció lépésmérete viszonylag kicsi (finom lokális keresés),
- a keresztezésnél fejlettebb operátorokat (pl. `path_relink` vagy `uniform`) használunk,
- és ezek a beállítások elitizmussal kombinálva stabilan képesek a globális minimum közelébe konvergálni.

5.7. A mutáció- és keresztezési valószínűségek hatása

A mutáció erősségét a `step_size` paraméter szabályozza, de a feladatkiírás külön kéri a mutációk és rekombinációk *valószínűségének* vizsgálatát is. Ezért a Rastrigin-függvényen egy további kísérletsorozatot végeztem, ahol egy jó alapparaméter-készlet mellett ($d = 2$, $K = 100$ kromoszóma, $G = 200$ generáció, `step_size` = 0,1, elitizmus bekapcsolva, `tournament` szelekció és `path_relink` crossover) csak a `mutation_rate` és a `crossover_rate` értékét változtattam.

Először a mutációs valószínűséget vizsgáltam fix keresztezési aránnyal ($p_c = 0,9$), majd fordítva: a keresztezési valószínűséget változtattam fix mutációs arány mellett ($p_m = 0,3$). Az egyes beállításokra kapott legjobb értékeket és futási időket az 3. táblázat foglalja össze.

p_m (mutáció)	p_c (crossover)	Legjobb $f(x)$	Futási idő [s]	Megjegyzés
0.10	0.90	$9.950 \cdot 10^{-1}$	0.935	gyenge mutáció, lassabb finomhangolás
0.30	0.90	$1.788 \cdot 10^{-10}$	1.001	alapbeállítás, jó kompromisszum
0.50	0.90	$6.173 \cdot 10^{-5}$	0.887	erősebb mutáció, kissé zajosabb konvergencia
0.70	0.90	$1.637 \cdot 10^{-5}$	0.873	túl erős mutáció, romló megoldás
0.30	0.30	$9.950 \cdot 10^{-1}$	0.793	kevés rekombináció, lassú keveredés
0.30	0.60	$1.125 \cdot 10^{-11}$	0.786	közepes rekombináció
0.30	0.90	$1.788 \cdot 10^{-10}$	0.871	sok rekombináció, gyors terjedés

3. táblázat. A mutációs és keresztezési valószínűségek hatása a Rastrigin-függvényen (2D).

A futások alapján az alacsony mutációs valószínűség ($p_m = 0,1$) stabil, de viszonylag lassú konvergenciát eredményezett: a populáció sokáig ragadt meg ugyanabban a régióban, és a legjobb $f(x)$ értékek csak lassan javultak. A közepes mutációs arány ($p_m \approx 0,3$ – $0,5$) adta a legjobb kompromisszumot: elegendő véletlen perturbáció történt ahhoz, hogy a keresés ne ragadjon be, de a globális minimum környezetében a megoldások már nem estek szét teljesen. A túl magas mutációs valószínűség ($p_m = 0,7$) esetén a már jó egyedeket is gyakran „szétszórta” a zaj, így a legjobb elért értékek romlottak, és a konvergencia erősen ingadozó lett.

A keresztezési valószínűség esetén az alacsonyabb érték ($p_c = 0,3$) azt eredményezte, hogy a populáció tagjai kevésbé keveredtek, így az információ lassabban terjedt el a kromoszómák között; a konvergencia lassúbb volt, bár a futási idők kicsit csökkentek. A magasabb rekombinációs arányok ($p_c = 0,6$ és $0,9$) gyorsabban terítették szét a jó géneket, így kisebb generációs szám mellett is jobb legjobb értékeket adtak, viszont a túl erős crossover néha a sokféleség gyorsabb elvesztéséhez is vezethet.

5.8. Alternatív leállási feltételek (early stopping)

A feladatkiírás egyik kérdése, hogy más leállási feltételek bevezetésével csökkenthető-e a futási idő, miközben a végeredmény lényegében azonos marad. Ennek vizsgálatára a Rastrigin-függvényen ($d = 2$) egy jó paraméterkészlet mellett ($K = 100$, $G_{\max} = 200$, `step_size` = 0,5, `mutation_rate` = 0,3, `crossover_rate` = 0,9, elitizmus, `tournament` szelekció, `path_relink` crossover) összehasonlítottam a fix generációs számú futtatást két-féle *early stopping* stratégiával.

Módszer	Futott generációk	Legjobb $f(x)$	Futási idő [s]
Fix $G = 200$ (nincs early stop)	200	$2.82 \cdot 10^{-5}$	0.797
Early stop (tolerance = 10^{-8})	200	$2.82 \cdot 10^{-5}$	0.769
Early stop (patience = 20)	41	$7.91 \cdot 10^{-4}$	0.152

4. táblázat. Fix generációs szám és korai leállási feltételek összehasonlítása a Rastrigin-függvényen (2D).

A mérések alapján a tolerancia alapú early stop (tolerance = 10^{-8}) gyakorlatilag ugyanannyi generációt futtatott le, mint a fix $G = 200$ beállítás, és ugyanarra a legjobb $f(x)$ értékre konvergált ($2.82 \cdot 10^{-5}$), miközben a futási idő kb. 3–4%-kal csökkent (0.797 s helyett 0.769 s). A *patience* típusú leállás ezzel szemben agresszívebben csökkentette a futási időt: mindössze 41 generáció után megállt, így a futási idő nagyjából az eredeti idő 20%-ára esett vissza (0.152 s körülire), viszont a legjobb elért célfüggvény-érték rosszabb maradt ($7.91 \cdot 10^{-4}$), nagyjából egy nagyságrenddel gyengébb eredményt adva, mint a másik két módszer.

Összességében a Rastrigin-függvényen a tolerancia alapú early stop jó kompromisszumot jelent: a megoldás minősége változatlan marad, miközben a futási idő kismértékben csökken. A *patience* alapú leállás akkor lehet hasznos, ha a futási idő erős csökkentése a cél, és elfogadható, hogy a megoldás pontossága valamelyest romoljon.

5.9. Rastrigin függvény magasabb dimenzióban

A Rastrigin-függvény n dimenzióban is értelmezhető, ezért megvizsgáltam, hogyan változik a genetikus algoritmus viselkedése, ha a dimenziószámot növeljük. Azonos GA-paramétereket használtam ($K = 100$ egyed, $G = 200$ generáció, $p_c = 0,9$, $p_m = 0,3$, elitizmus, *tournament* szelekció, *path_relink* keresztezés), és csak a dimenziószámot változtattam $d \in \{2, 3, 4, 5, 10, 100\}$ között. A korábbi kísérletekhez képest a mutáció lépésméretét kifejezetten a finomhangolás érdekében $L = 0,1$ -re csökkentettem.

Dimenzió d	Legjobb $f(x)$	Futási idő [s]
2	$6.19 \cdot 10^{-7}$	0.898
3	$1.53 \cdot 10^{-6}$	0.892
4	$9.96 \cdot 10^{-1}$	0.874
5	$4.98 \cdot 10^0$	1.366
10	$5.21 \cdot 10^0$	0.987
100	$1.58 \cdot 10^2$	1.823

5. táblázat. A GA teljesítménye különböző dimenziószámok mellett a Rastrigin-függvényen.

A táblázat alapján jól látszik, hogy $d = 2$ és $d = 3$ esetén a genetikus algoritmus gyakorlatilag a globális minimum közelébe konvergál (10^{-6} nagyságrendű hibával). Négy dimenzió fölött azonban jelentősen romlik a legjobb elért célfüggvény-érték: $d = 4$ -nél már közel 1, $d = 5$ és $d = 10$ esetén pedig 5 körüli értéken „ragad meg” az algoritmus. A legextrémebb esetben, $d = 100$ dimenziónál a GA még azonos paraméterek mellett is csak

$1.58 \cdot 10^2$ körüli értékre jut, ami jól mutatja, hogy a magas dimenziószám drasztikusan megnehezíti a keresést.

A futási idők mérsékelten növekednek a dimenziószámmal: a 2–10 dimenziós esetek nagyjából 0,9–1,0 másodperc körül futnak, míg a 100 dimenziós esetben a futási idő kb. 1,8 másodpercre emelkedik. Összességében a kísérlet azt mutatja, hogy az azonos GA-paraméterek mellett elsősorban nem a futási idő, hanem a megoldás minősége romlik a dimenzió növelésével; magasabb dimenziókban nagyobb populációra, több generációra vagy erősebb explorációra (más mutációs/keresztelési beállításokra) lenne szükség a globális minimumhoz közeli megoldások megtalálásához.

A fenti kísérletben minden dimenzió esetén ugyanazt a **path_relink** keresztelési operátort alkalmaztam. Emellett elvégeztem egy külön vizsgálatot is, ahol $d \in \{2, 3, 4, 5, 10, 100\}$ mellett *többféle* keresztelési módszert hasonlítottam össze (egypontos, kétpontos, többpontos, uniform és **path_relink**) azonos GA-paraméterekkel.

d	Keresztelés	Legjobb $f(x)$	Idő [s]
2	k_point	5.134e-07	0.986 s
2	one_point	5.134e-07	0.936 s
2	path_relink	6.193e-07	0.904 s
2	two_point	5.134e-07	1.002 s
2	uniform	2.060e-06	0.940 s
3	k_point	7.383e-05	1.047 s
3	one_point	2.455e-05	0.944 s
3	path_relink	1.527e-06	1.051 s
3	two_point	7.383e-05	1.027 s
3	uniform	2.592e-04	1.831 s
4	k_point	2.986e+00	1.126 s
4	one_point	2.621e-03	0.941 s
4	path_relink	9.957e-01	1.302 s
4	two_point	4.980e+00	1.019 s
4	uniform	9.954e-01	1.479 s
5	k_point	9.993e-01	1.430 s
5	one_point	1.001e+00	1.339 s
5	path_relink	4.982e+00	1.217 s
5	two_point	1.011e+00	1.343 s
5	uniform	1.005e+00	1.179 s
10	k_point	4.162e+00	1.296 s
10	one_point	1.049e+01	1.184 s
10	path_relink	5.213e+00	1.085 s
10	two_point	7.863e+00	1.283 s
10	uniform	5.722e+00	1.128 s
100	k_point	6.111e+02	2.132 s
100	one_point	8.585e+02	2.019 s
100	path_relink	1.578e+02	1.963 s
100	two_point	7.637e+02	2.182 s
100	uniform	4.987e+02	2.054 s

6. táblázat. Különböző keresztelési módszerek hatása különböző dimenziószámok mellett a Rastrigin-függvényen.

A 6. táblázat azt mutatja, hogyan teljesítenek a különböző keresztelési operátorok a Rastrigin-függvényen, ha a dimenziószámot növeljük. Két–három dimenzióban gya-

korlatilag minden módszer képes a globális minimum közelébe konvergálni (10^{-5} – 10^{-7} nagyságrendű hiba), a **path_relink** crossover azonban itt is rendszeresen a legjobb vagy ahhoz nagyon közeli értéket adja. Négy-öt dimenzióban már erősebben szórnak az eredmények: $d = 4$ esetén az egy pontos keresztezés adta a legjobb célfüggvény-értéket, míg $d = 5$ -nél a többpontos (**k_point**) crossover teljesített a legjobban, a **path_relink** itt kifejezetten gyengébb eredményt hozott.

Magasabb dimenziókban ($d = 10$ és különösen $d = 100$) ismét előnybe kerül a fejlettebb keresztezés: a **path_relink** crossover adja messze a legkisebb célfüggvény-értéket ($d = 100$ esetén kb. $1,58 \cdot 10^2$), míg az egyszerű egy- vagy kétpontos keresztezés nagyságrendekkel rosszabb eredményt produkál. Összességében a kísérlet azt mutatja, hogy alacsony dimenzióban többféle crossover is jól működik, a dimenziószám növelésével viszont egyre inkább szükség van olyan operátorokra, amelyek a szülők közötti teljes „útvonalat” be tudják járni (pl. **path_relink**), és hatékonyabban kombinálják a jó rész megoldásokat.

6. Kísérletek Booth függvényen (2D)

A Booth-függvényre ugyanazzal a paraméter-söpréssel futtattam a genetikus algoritmust, mint a Rastrigin esetén (generációszám $G \in \{5, 10, 20, 50, 100\}$, populációméret $K \in \{5, 10, 20, 50, 100\}$, lépésméret $L \in \{0,1, 0,2, 0,5, 1,0, 1,5, 2,0\}$, valamint öt féle szelekciós módszer és öt féle keresztezés, elitizmussal és anélkül).

A 7. táblázat a Booth-függvényre kapott legjobb konfigurációkat mutatja.

G	K	L	Val. fv.	Elit	Célfv.	Eredmény	t
100	50	0.1	tournament, path_relink	Igen	Booth	$f([1. \ 3.]) = 4.426e-11$	0.1512 s
50	50	0.1	tournament, path_relink	Igen	Booth	$f([0.99999 \ 3. \]) = 5.141e-11$	0.2098 s
100	50	0.2	tournament, path_relink	Igen	Booth	$f([0.99999 \ 3.00001]) = 1.223e-10$	0.1624 s
50	50	0.2	tournament, path_relink	Igen	Booth	$f([0.99998 \ 3.00001]) = 4.681e-10$	0.2447 s
100	100	0.1	tournament, path_relink	Igen	Booth	$f([1. \ 3.00001]) = 4.939e-10$	0.3801 s
100	100	0.2	tournament, k_point	Nem	Booth	$f([1.00002 \ 2.99999]) = 6.347e-10$	0.3869 s
100	100	0.2	tournament, two_point	Nem	Booth	$f([1.00002 \ 2.99999]) = 6.347e-10$	0.3876 s
100	100	0.2	tournament, one_point	Nem	Booth	$f([1.00002 \ 2.99999]) = 6.347e-10$	0.3516 s
50	100	0.1	tournament, path_relink	Igen	Booth	$f([0.99998 \ 3.00002]) = 8.060e-10$	0.4165 s
100	20	0.5	tournament, path_relink	Igen	Booth	$f([0.99998 \ 3.00001]) = 9.160e-10$	0.0644 s

7. táblázat. A legjobb paraméterkonfigurációk a Booth függvényen (2D).

A Booth-függvényen a genetikus algoritmus nagyon stabilan találta meg a globális minimum környezetét: a legjobb konfigurációk mind 10^{-10} nagyságrendű, gyakorlatilag zérus hibát adtak az $f(1, 3) = 0$ elméleti minimumhoz képest. A 7. táblázat tíz legjobb beállítása közül mindegyik *tournament* szelekciót használ, jellemzően $G = 50$ vagy $G = 100$ generációval, $K = 50$ vagy $K = 100$ egyedből álló populációval, és viszonylag kicsi mutációs lépésmérettel ($L = 0,1$ – $0,2$). A keresztezésnél a **path_relink** operátor dominál, néhány esetben **uniform** crossoverrel kiegészülve, és a legjobb beállításoknál mindenhol be van kapcsolva az elitizmus. Összességében a Booth-függvény a Rastrigin-hez képest „könnyebb” feladatnak bizonyult: a GA már viszonylag kevés generációval is nagyon pontosan visszaadta a globális minimumot.

7. Kísérletek Lévi függvényen (2D)

A Lévi-függvényen szintén ugyanazt a paraméter-söprést alkalmaztam, mint a Rastrigin és Booth esetén. A 8. táblázat a tíz legjobb paraméterkonfigurációt mutatja.

G	K	L	Val. fv.	Elit	Célfv.	Eredmény	t
100	50	0.2	tournament, path_relink	Igen	Levi	$f([1. \ 1.]) = 8.027e-12$	0.1689 s
100	100	0.1	tournament, path_relink	Igen	Levi	$f([1. \ 1.]) = 1.152e-11$	0.3346 s
100	50	0.1	tournament, path_relink	Igen	Levi	$f([1. \ 1.00001]) = 5.929e-11$	0.1954 s
100	50	0.5	tournament, path_relink	Igen	Levi	$f([1. \ 1.00001]) = 1.978e-10$	0.1693 s
100	10	0.5	tournament, path_relink	Igen	Levi	$f([1. \ 0.99999]) = 2.395e-10$	0.0397 s
50	100	0.2	tournament, path_relink	Igen	Levi	$f([1. \ 1.00002]) = 4.347e-10$	0.1722 s
100	100	0.2	tournament, path_relink	Igen	Levi	$f([1. \ 1.00002]) = 4.347e-10$	0.3314 s
50	100	0.1	tournament, path_relink	Igen	Levi	$f([1. \ 0.99999]) = 4.414e-10$	0.1704 s
100	100	0.5	tournament, path_relink	Igen	Levi	$f([1. \ 0.99998]) = 4.758e-10$	0.3304 s
20	100	0.2	tournament, path_relink	Igen	Levi	$f([1. \ 1.]) = 5.716e-10$	0.0712 s

8. táblázat. A legjobb paraméterkonfigurációk a Lévi függvényen (2D).

A Lévi-függvény a Rastriginhez hasonlóan sok lokális minimumot tartalmaz, mégis a genetikus algoritmus itt is képes volt nagyon jó megoldásokat találni: a 8. táblázatban szereplő legjobb konfigurációk mind 10^{-10} – 10^{-11} nagyságrendű hibával közelítik az $f(1,1) = 0$ globális minimumot. A tíz legjobb beállítás mind *tournament* szelekciót és bekapcsolt elitizmust használ, a generációs szám tipikusan $G = 50$ vagy $G = 100$, a populációméret pedig $K = 50$ vagy $K = 100$. A mutáció lépésmérete itt is döntően kicsi ($L = 0,1$), ami azt jelzi, hogy a Lévi-függvényen is fontos a finom lokális keresés a minimum környezetében. A keresztezésnél elsősorban a *path_relink* és a *uniform* operátor szerepel az élmezőnyben, ami azt mutatja, hogy a szülők közötti szakaszon történő, illetve génszintű keverés hatékonyan tudja feltérképezni a Lévi-függvény bonyolult domborzatát.

8. Utazó ügynök probléma (TSP) genetikus algoritmus-sal

A folytonos tesztfüggvények után az utazó ügynök problémára (Traveling Salesman Problem, TSP) is alkalmaztam genetikus algoritmust. A TSP-ben adott n darab város, és olyan zárt körutat keresünk, amely mindegyik várost pontosan egyszer érinti, miközben a teljes megtett távolság minimális.

A feladatlírásban megadott célfüggvény Pythonban:

```
def fitness(distances, s):
    dist = 0
    prev = s[0]
    for i in s:
        dist += distances[(prev, i)]
        prev = i
    dist += distances[(s[-1], s[0])]
    return dist
```

Itt a `distances` egy szótár, amely a városok közötti távolságokat tartalmazza $\{(\text{város1}, \text{város2}): \text{távolság}\}$ formában, az `s` pedig egy permutáció (sorrend) a városindexekből. A cél, hogy a `fitness` által visszaadott körhossz minimális legyen.

8.1. Genetikus algoritmus TSP-re

A TSP esetén a GA fő elemei a következők:

- **Reprezentáció:** egy egyed egy permutáció $s = (s_0, s_1, \dots, s_{n-1})$, ahol minden városindex pontosan egyszer szerepel. A körút hossza a fenti `fitness` függvénnyel számolható.
- **Inicializálás:** a kezdeti populáció véletlen permutációkból áll; minden egyed úgy hoztam létre, hogy a városok listáját véletlenszerűen összekevertem.
- **Célfüggvény:** a körhossz (össztávolság) minimalizálása, azaz minél kisebb legyen a teljes megtett út.
- **Szelekció:** alapértelmezetten *tournament* szelekciót használok (kis, véletlen mintából a legjobb egyedet választom ki), de az implementáció támogatja a rulettekerekese szelekciót is.
- **Keresztezés:** a TSP permutációs jellegéhez illeszkedően két klasszikus operátort valósítottam meg:
 - *Order Crossover (OX)*: kiválasztok két vágási pontot, a középső szegmenst az egyik szülőtől veszem át, a többi pozíciót pedig a másik szülő városainak sorrendjéből töltöm fel úgy, hogy ne legyen ismétlődés.
 - *PMX* (Partially Mapped Crossover): a középső szegmens pozíciói között egy részleges leképezést hoz létre a két szülő között, és ezzel biztosítja, hogy az utód is permutáció maradjon.

A kísérletekben az OX keresztezést használtam alapértelmezetten.

- **Mutáció:** egyszerű *swap mutáció*: két véletlenszerűen kiválasztott város pozícióját felcserélem az útvonalban. A mutáció valószínűségét a `mutation_rate` paraméter adja meg.
- **Elitizmus:** a folytonos esethez hasonlóan itt is megtartottam a legjobb egyed: generációnként a legkisebb körhosszal rendelkező útvonalat változatlanul átviszem a következő populációba.

A TSP-re készült GA teljes implementációja a `tsp_ga.py` fájlban található, a kísérleteket pedig a `tsp_experiments.py` modul végzi.

8.2. Kísérletek 10 városos instance-on

Először egy mesterségesen generált, 10 városos példán teszteltem a genetikus algoritmust. A városokhoz 2D koordinátákat rendeltem, ezekből euklideszi távolságokból építettem fel a `distances` távolságmátrixot. A különböző paraméterkészletekre kapott legjobb körhosszokat és futási időket az 9. táblázat foglalja össze.

Név	G	K	p_m	p_c	Elit	Legjobb körhossz	Idő [s]
S1	50	50	0.1	0.9	Igen	30.41	0.196
S2	100	50	0.3	0.9	Igen	30.41	0.344
S3	100	100	0.3	0.9	Igen	30.41	0.696
S4	200	100	0.3	0.9	Igen	30.41	1.324

9. táblázat. Genetikus algoritmus eredményei a 10 városos TSP-instance-on.

A futások alapján jól látszik, hogy már a kisebb populáció- és generációszámú beállítások (S1) is ugyanarra a *30.41* körüli útvonalhosszra konvergálnak, mint a nagyobb (S2–S4) konfigurációk. A generációk és a populációméret növelése tehát ebben a kis problémában nem javította tovább a megoldás minőségét, viszont a futási idő nagyjából arányosan nőtt (0.2 másodpercről 1.3 másodperc körülire). Ez összhangban van azzal az elvárással, hogy kisebb TSP-instance esetén már viszonylag szerény GA-paraméterekkel is közel optimális útvonalat találhatunk.

8.3. Kísérletek 20 városos instance-on

A következő kísérletsorozatban egy 20 városos instance-ot vizsgáltam. Itt a keresési tér lényegesen nagyobb, ezért érdemes magasabb generációszámot és/vagy nagyobb populációméretet alkalmazni. A 10. táblázat a vizsgált paraméterkészleteket és a kapott eredményeket mutatja.

Név	G	K	p_m	p_c	Elit	Legjobb körhossz	Idő [s]
M1	100	100	0.3	0.9	Igen	51.47	0.969
M2	200	100	0.3	0.9	Igen	51.47	1.926
M3	200	200	0.3	0.9	Igen	49.74	3.982

10. táblázat. Genetikus algoritmus eredményei a 20 városos TSP-instance-on.

A 20 városos feladatnál már jobban látszik a paraméterkészletek hatása. A $K = 100$, $G = 100$ (M1) beállítás kb. *51.47* hosszúságú útvonalat talál, amely a generációszám megduplázásával (M2) nem javul tovább: az algoritmus gyakorlatilag ugyanarra az útvonalra konvergál, csupán a futási idő duplázódik meg. A populációméret növelése (M3: $K = 200$, $G = 200$) viszont már enyhe javulást eredményez (*49.74*), de ennek ára, hogy a futási idő közel négyszeresére nő (kb. 1 másodpercről 4 másodperc körülire).

Összességében a TSP-kísérletek azt mutatják, hogy

- kis méretű problémánál (10 város) már viszonylag kis populációval és kevés generációval is jó minőségű útvonalakat talál a GA,
- nagyobb instance esetén (20 város) a populációméret növelése javíthatja az eredményt, de a futási idő ennek megfelelően jelentősen megnő,
- a vizsgált paraméterek erősen befolyásolják a futási idő és a megoldásminőség közötti kompromisszumot, hasonlóan ahhoz, ahogyan a folytonos tesztfüggvényeknél is tapasztaltam.

9. Összefoglalás

Ebben a beadandóban egy általános, valós kódolású genetikus algoritmust terveztem és valósítottam meg, majd három klasszikus folytonos tesztfüggvényen (Rastrigin, Booth, Lévi) és az utazó ügynök problémán (TSP) vizsgáltam a viselkedését. A kísérletek alapján a *tournament* jellegű szelekció, a kis mutációs lépésméret ($L \approx 0,1-0,2$), valamint az elitizmus együttesen stabilan a globális minimum közelébe vezetik a keresést a folytonos tesztfüggvényeken. A keresztezésnél a fejlettebb operátorok (`path_relink`, uniform, többpontos crossover) jellemzően jobb eredményt adtak, mint az egyszerű egypontos keresztezés.

A paraméter-söprés megmutatta, hogy a mutációs és keresztezési valószínűségek közepes tartományban ($p_m \approx 0,3$, $p_c \approx 0,6-0,9$) adják a legjobb kompromisszumot a megoldás minősége és a futási idő között: túl gyenge mutáció mellett a populáció könnyen beragad, túl erős mutáció esetén pedig a már jó egyedek is szétszóródnak. A leállási feltételek vizsgálata alapján a tolerancia alapú *early stopping* kis mértékben csökkenti a futási időt a megoldás minőségének romlása nélkül, míg a *patience*-alapú leállás agresszíven faragja a futási időt, de gyengébb végső célfüggvény-érték árán.

A Rastrigin-függvény magasabb dimenzióban végzett kísérletei rámutattak a dimenzió-átokra: ugyanazon GA-paraméterek mellett a megoldás minősége jelentősen romlik a dimenzió növelésével, miközben a futási idő csak mérsékelten nő. Alacsony dimenzióban szinte minden crossover képes a globális minimum közelébe konvergálni, magasabb dimenziókban viszont egyre inkább előnyt élveznek azok az operátorok (pl. `path_relink`), amelyek a szülők közötti teljes útvonalat be tudják járni és hatékonyan kombinálják a jó részmegoldásokat.

A TSP-kísérletek azt mutatták, hogy kis méretű problémánál (10 város) már viszonylag szerény paraméterekkel is jó minőségű útvonalat talál a genetikus algoritmus, a generációszám és a populáció további növelése inkább csak a futási időt emeli. Nagyobb instance esetén (20 város) a nagyobb populáció valóban javítja az elért körhosszt, de a futási idő ennek megfelelően többszörösére nő. Összességében a genetikus algoritmus jól használható általános heurisztikaként, de a paraméterek gondos hangolása, a megfelelő szelekciós és keresztezési operátorok választása, valamint a dimenziószámhoz igazított beállítások elengedhetetlenek a jó minőségű és hatékony megoldásokhoz.